

1

The screenshot shows the LeetCode interface for the problem "Letter Combinations of a Phone Number". The problem description includes a phone keypad and examples of input digits and their corresponding letter combinations. The constraints specify the length of the digits and the range of digits. The solution is implemented in C++ using a backtracking function. The test result shows the solution is accepted.

Problem Description:

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Mapping of digit to letters (as on a telephone keyboard) is shown below.

Example 1:

Input: digits = "23"

Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Example 2:

Input: digits = ""

Output: []

Constraints:

- 1 ≤ digits.length ≤ 4
- digits[i] is a digit in the range [2, 9].

Accepted: 2,925,866 / 4,5M **Acceptance Rate:** 65.5%

Code:

```

1  class Solution {
2  public:
3      vector<string> result;
4      string current;
5      void backtrack(const string& digits, int index, string current, vector<string>& result) {
6          if (index == digits.length()) {
7              result.push_back(current);
8              return;
9          }
10         string letters = mappings[digits[index] - '0'];
11         for (char c : letters) {
12             current.push_back(c);
13             backtrack(digits, index + 1, current, result);
14             current.pop_back();
15         }
16     }
17     vector<string> letterCombinations(string digits) {
18         backtrack(digits, 0, "", result);
19         return result;
20     }
21 };

```

Testcase: Accepted Runtime: 0 ms

Case 1:

Input: digits = "23"

Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

Expected: ["ad","ae","af","bd","be","bf","cd","ce","cf"]

2

The screenshot shows the LeetCode interface for the problem "4Sum". The problem description includes the goal of finding unique quadruplets that sum to a target. The constraints specify the range of the numbers and the target. The solution is implemented in C++ using a two-pointer approach. The test result shows the solution is accepted.

Problem Description:

Given an array `nums` of `n` integers, return an array of all the **unique** quadruplets `[nums[a], nums[b], nums[c], nums[d]]` such that:

- 0 ≤ a, b, c, d < n
- nums[a], nums[b], nums[c], and nums[d] are distinct.
- nums[a] + nums[b] + nums[c] + nums[d] == target

You may return the answer in **any order**.

Example 1:

Input: nums = [1,0,-1,0,-2,2], target = 0

Output: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]

Example 2:

Input: nums = [2,2,2,2,2], target = 8

Output: [[2,2,2,2]]

Constraints:

- 1 ≤ nums.length ≤ 200
- 10⁵ ≤ nums[i] ≤ 10⁵
- 10⁵ ≤ target ≤ 10⁵

Accepted: 1,608,243 / 4M **Acceptance Rate:** 40.0%

Code:

```

1  class Solution {
2  public:
3      vector<vector<int>> result;
4      long long sum = (long long)nums[0] + nums[1] + nums[2] + nums[3];
5      void findQuadruplets(int left, int right, int target) {
6          if (sum == target) {
7              result.push_back({nums[left], nums[right], nums[left+1], nums[right+1]});
8              while (left < right && nums[left] == nums[left+1]) left++;
9              while (left < right && nums[right] == nums[right-1]) right--;
10             left++;
11             right--;
12         }
13         else if (sum < target) {
14             left++;
15         }
16         else {
17             right--;
18         }
19     }
20     vector<vector<int>> fourSum(vector<int>& nums, int target) {
21         sort(nums.begin(), nums.end());
22         for (int i = 0; i < nums.size(); i++) {
23             for (int j = i + 1; j < nums.size(); j++) {
24                 findQuadruplets(j, nums.size() - 1, target - nums[i] - nums[j]);
25             }
26         }
27         return result;
28     }
29 };

```

Testcase: Accepted Runtime: 0 ms

Case 1:

Input: nums = [1,0,-1,0,-2,2], target = 0

Output: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]

Expected: [[-2,-1,1,2],[-2,0,0,2],[-1,0,0,1]]

3

Screenshot of the LeetCode problem page for "Remove Nth Node From End of List". The problem description shows Example 2: Input: head = [1], n = 1, Output: []. Example 3: Input: head = [1,2], n = 1, Output: [1]. Constraints: The number of nodes in the list is sz . $1 \leq sz \leq 30$. $0 \leq \text{Node.val} \leq 100$. $1 \leq n \leq sz$. Follow up: Could you do this in one pass? The solution code is in C++ and uses a two-pointer approach to remove the nth node from the end of the list. The test result shows "Accepted" with a runtime of 0 ms for Case 1.

4

Screenshot of the LeetCode problem page for "Valid Parentheses". The problem description shows Example 5: Input: s = "()", Output: true. Constraints: $1 \leq s.length \leq 10^4$. s consists of parentheses only '(){}[]'. The solution code is in C++ and uses a stack to validate the parentheses. The test result shows "Accepted" with a runtime of 0 ms for Case 1.

5

leetcode.com/problems/merge-two-sorted-lists/

Description | Editorial | Solutions | Submissions

Merge the two lists into one **sorted** list. The list should be made by splicing together the nodes of the first two lists.
Return the head of the merged linked list.

Example 1:

Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]

Example 2:

Input: list1 = [], list2 = []
Output: []

Example 3:

Input: list1 = [], list2 = [0]
Output: [0]

Constraints:

- The number of nodes in both lists is in the range $[0, 50]$.
- $-100 \leq \text{Node.val} \leq 100$
- Both `list1` and `list2` are sorted in **non-decreasing** order.

Seen this question in a real interview before? 1/5

24.8K 580 381 Online

Code

```

C++
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        if (!list1) return list2;
        if (!list2) return list1;
        if (list1->val < list2->val) {
            list1->next = mergeTwoLists(list1->next, list2);
            return list1;
        } else {
            list2->next = mergeTwoLists(list1, list2->next);
            return list2;
        }
    }
};

```

Testcase | **Test Result**

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

list1 = [1,2,4]

list2 = [1,3,4]

Output

[1,1,2,3,4,4]

Expected

[1,1,2,3,4,4]

6

leetcode.com/problems/generate-parentheses/

22. Generate Parentheses

Medium | Topics | Companies

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Example 1:

Input: $n = 3$
Output: ["((()))","(()())","(())()","()(())","()()()"]

Example 2:

Input: $n = 1$
Output: ["()"]

Constraints:

- $1 \leq n \leq 8$

Seen this question in a real interview before? 1/5

Accepted 2,782,937 3.6M Acceptance Rate 78.3%

Topics Companies Similar Questions Discussion (258)

Copyright © 2026 LeetCode. All rights reserved.

23.1K 258 154 Online

Code

```

C++
class Solution {
public:
    vector<string> generateParenthesis(int n) {
        vector<string> result;
        backtrack(result, "", 0, 0, n);
        return result;
    }
private:
    void backtrack(vector<string>& result, string current, int open, int close, int n) {
        if (current.length() == n * 2) {
            result.push_back(current);
            return;
        }
        if (open < n) {
            current.push_back('(');
            backtrack(result, current, open + 1, close, n);
            current.pop_back();
        }
        if (close < open) {
            current.push_back(')');
            backtrack(result, current, open, close + 1, n);
            current.pop_back();
        }
    }
};

```

Testcase | **Test Result**

Accepted Runtime: 0 ms

Case 1 Case 2

Input

n = 3

Output

["((()))","(()())","(())()","()(())","()()()"]

Expected

["((()))","(()())","(())()","()(())","()()()"]

7

23. Merge k Sorted Lists

You are given an array of k linked-lists `lists`, each linked-list is sorted in ascending order. Merge all the linked-lists into one sorted linked-list and return it.

Example 1:
Input: `lists = [[1,4,5],[1,3,4],[2,6]]`
Output: `[1,1,2,3,4,4,5,6]`
Explanation: The linked-lists are:
1 → 4 → 5,
1 → 3 → 4,
2 → 6
merging them into one sorted Linked List:
1 → 1 → 2 → 3 → 4 → 4 → 5 → 6

Example 2:
Input: `lists = []`
Output: `[]`

Example 3:
Input: `lists = [[]]`
Output: `[]`

Constraints:

- $k == \text{lists.length}$
- $0 \leq k \leq 10^4$
- $0 \leq \text{lists[i].length} \leq 500$
- $-10^4 \leq \text{lists[i][j]} \leq 10^4$
- `lists[i]` is sorted in ascending order.
- The sum of `lists[i].length` will not exceed 10^4 .

```
1 for (ListNode* list : lists) {
2     if (list != nullptr) {
3         pq.push(list);
4     }
5 }
6
7 ListNode dummy(0);
8 ListNode* current = &dummy;
9
10 while (!pq.empty()) {
11     ListNode* smallest = pq.top();
12     pq.pop();
13
14     current->next = smallest;
15     current = current->next;
16
17     if (smallest->next != nullptr) {
18         pq.push(smallest->next);
19     }
20 }
21
22 return dummy->next;
```

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input: `lists = [[1,4,5],[1,3,4],[2,6]]`

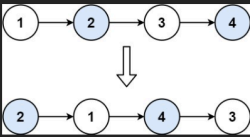
Output: `[1,1,2,3,4,4,5,6]`

Expected: `[1,1,2,3,4,4,5,6]`

8

24. Swap Nodes in Pairs

Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed).

Example 1:
Input: `head = [1,2,3,4]`
Output: `[2,1,4,3]`
Explanation:


Example 2:
Input: `head = []`
Output: `[]`

Example 3:
Input: `head = [1]`
Output: `[1]`

Example 4:
Input: `head = [1,2,3]`
Output: `[2,1,3]`

```
1 // Swap two adjacent nodes
2 // head is the pointer to the first node
3 // head->next is the pointer to the second node
4 if (head == nullptr || head->next == nullptr) {
5     return head;
6 }
7
8 ListNode dummy(0);
9 dummy->next = head;
10 ListNode* prev = &dummy;
11 ListNode* current = head;
12
13 while (current != nullptr && current->next != nullptr) {
14     ListNode* nextPair = current->next->next;
15     ListNode* second = current->next;
16
17     second->next = current;
18     current->next = nextPair;
19     prev->next = second;
20
21     prev = current;
22     current = nextPair;
23 }
24
25 return dummy->next;
```

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3 Case 4

Input: `head = [1,2,3,4]`

Output: `[2,1,4,3]`

Expected: `[2,1,4,3]`